# Payments Admin CRM — Business Case & Feature Breakdown

**Prepared by:** Keith Curbelo, Senior Engineer — Payments
**Audience:** Product Leadership
**Purpose:** Secure buy-in to move from proof-of-concept to production
**Target:** Buy-in within Q3 · Lightweight production release by end of year

---

**Application naming note:** The checkout microfrontend is currently named **accessoPay** and will be rebranded to **accesso checkout** in the near future. This document uses accessoPay throughout and will be updated once the rebrand is finalized.

---

## The Problem We're Solving

Client requests that touch the payment application fall into two categories today, and both carry more engineering overhead than they should.

**Category 1 — Enabling or adjusting an existing configuration**

When a client wants to enable a payment method that is already supported, adjust their layout, or toggle a feature that already exists in the system, this does not go through the full SDLC. It goes through a SQL deployment process: a deployment task is created and a skilled engineer must write and execute the SQL against the production database directly. It requires engineering time, carries execution risk, and depends on the availability of someone qualified to run it safely. There is no interface, no audit trail, and no way to preview or reverse the change without another deployment.

**Category 2 — Integrating a new payment method or provider**

When a client requests a payment method or provider that does not yet exist in the application, that work enters the full software development lifecycle:

- The PO holds meetings to gather requirements and align with engineering
- Engineering reviews the integration, writes stories, and estimates effort
- The work is prioritized into a sprint
- Developers build the integration, touching critical application and configuration code
- Code review, QA, and deployment follow

**This takes weeks and consumes sprint capacity that should go toward new product features.**

In both cases, the underlying problem is the same: there is no dedicated, safe interface for managing how the payment application behaves per client. Routine configuration work requires direct engineering involvement, carries risk, and leaves no visible record of what was changed or why.

We are also spread across multiple stacks, tenants, and hundreds of merchants with no centralized view. When something goes wrong, the team has to manually dig through systems to understand what a merchant is configured to run, what changed, and when.

---

## What We're Building

The **Payments Admin CRM** is a dedicated, role-based management platform built exclusively for the payments domain. It gives the Product team, Operations, and Production Support the ability to safely view,

configure, test, and deploy merchant settings — without writing code or going through the SDLC.

It is **not** a generic tool. It is purpose-built around our payment application's schema, our stacks, our integration partners, and our team's workflows.

Think of it as a control panel for the payment application — one that replaces fragmented processes, multiple tools, and direct engineering involvement for routine configuration work.

---

## Features — What They Are & What They Require

Each feature below is broken down by what it does for the business and what it requires from engineering to build in production.

---

### 1. Merchant Configuration Management

**What it does**

Provides a visual interface to view and edit the full configuration of any merchant — theme and branding, which pages are enabled, which components appear on each page, which integrations are active, and which feature flags are turned on or off.

Today, enabling or adjusting a merchant's configuration requires a SQL deployment task executed by a skilled engineer directly against the production database. There is no interface, no validation, and no record of the change beyond what the engineer manually documents. This feature replaces that process entirely with a visual, validated, audited interface that non-engineers can use safely.

**Business value**

- Operations and Product can enable and adjust merchant configurations without engineering involvement or SQL deployments
- Eliminates execution risk — the UI enforces valid configuration rules and prevents invalid states from being saved
- Every change is logged automatically, replacing the current lack of any audit trail

**Engineering requirements**

- Connect the CRM to AccessoPayService via a new aggregation endpoint that serves live merchant configs
- Build the admin overlay write path — when a change is saved in the CRM, it needs to persist to a separate store that gets merged with the base config at request time
- Role-based access: read-only vs. edit permissions per user type
- Input validation to prevent invalid configs from being saved

---

### 2. Live Preview Mode

**What it does**

Before publishing any change, the user can see exactly how the checkout experience will look and behave for the merchant — including theme colors, which payment methods appear, which pages are in the flow, and which components are visible. Changes to the configuration update the preview in real time.

**Business value**

- Gives Operations and Product confidence before pushing a change live
- Eliminates the need to deploy to a staging environment just to see what a change looks like

- Reduces the back-and-forth between client requests and engineering validation

**Engineering requirements**

- Preview iframe or in-CRM mockup stays in sync with the editing state (already proven in POC)
- "Preview App" button opens a real instance of accessoPay loaded with the draft config, scoped to the specific tenant and merchant
- The checkout app (accessoPay) needs a config-loading mechanism that can accept the draft config from a secure CRM-controlled source

---

## 3. Config Version History & Rollback

**What it does**

Every time a configuration is published, a snapshot is saved with a timestamp and author. The team can browse the full history of changes for any merchant and restore any previous version with a single click.

**Business value**

- Eliminates fear of making changes — any mistake can be undone immediately
- Gives the team a complete audit trail: who changed what, when, and from what value
- Reduces incident response time when a misconfiguration causes a client issue — no need to dig through code history or deployment logs

**Engineering requirements**

- Version snapshots need to be persisted server-side (database), not just in browser memory
- Restore action must write back through the same overlay write path as a normal publish
- Audit log entries need to capture user identity (tied to role-based auth)
- Retention policy decision: how many versions to keep per merchant

---

## 4. Release & Migration Center

**What it does**

As the payment application evolves — new features, new schema fields, new integrations — merchants need to be migrated to new versions. Today this process is undocumented and manual: engineers figure out what changed, what each merchant needs, and make the updates by hand.

The Release & Migration Center introduces a structured process. When engineers cut a new version of the payment application, they publish a GitHub release with a defined template that documents every schema change, what it means, and which merchants are affected. The CRM reads these releases automatically and presents:

- A changelog of what changed in each version and why
- Per-merchant migration status: which merchants are on which version, what changes they need
- Guided migration steps so the team can apply changes safely, one merchant at a time

**Business value**

- Eliminates hours of manual investigation each time the application ships a new version
- Gives Product and Ops visibility into migration status across all merchants without asking engineering
- Creates a documented history of schema evolution that currently does not exist
- Reduces the chance of a merchant being accidentally left on an incompatible version

**Engineering requirements**

- Engineers fill in a structured GitHub release template when cutting each version (minimal additional effort — template is already built)
- CRM fetches releases via GitHub API and parses the structured markdown into migration guidance
- Migration status tracking per merchant needs to be persisted server-side
- Future: automated migration suggestions or one-click migration scripts where changes are low-risk

---

## 5. Analytics Dashboard

**What it does**

Provides high-level operational insights across merchants and stacks — configuration adoption rates, which features are enabled across the fleet, comparison views between merchants or tenants.

In a future iteration this can be connected to real transaction and conversion data to show business-level impact of configuration choices (e.g. "merchants with Google Pay enabled show X% higher conversion").

**Business value**

- Product team gains a portfolio view of the payment application without asking engineering for reports
- Side-by-side merchant comparison helps identify inconsistencies and opportunities
- Data to support client conversations about which features to adopt

**Engineering requirements**

- Initial phase: derive analytics from config data already available (no new data sources needed)
- Future phase: connect to transaction/event data sources
- Aggregation queries need to be efficient across potentially hundreds of merchants
- Caching strategy to avoid excessive backend calls

---

## 6. Logs & Observability

**What it does**

A searchable, filterable log view scoped to the payments domain — configuration changes, publish events, integration health, system errors. Logs can be filtered by merchant, stack, service, severity, and date range.

Today, investigating a production issue requires engineers to manually query multiple systems. This brings relevant logs into one place for the team.

**Business value**

- Production Support can triage client issues without pulling in an engineer for log access
- Operations has visibility into configuration publish history and system events
- Reduces time-to-resolution for incidents

**Engineering requirements**

- Connect to existing log infrastructure (no new logging needed — aggregation only)
- Role-based visibility: Production Support sees merchant-level logs; engineers see system-level
- Search and filter performance at volume
- Log retention aligned with existing data policy

---

## 7. Role-Based Access Control (RBAC)

**What it does**

Different users get different levels of access based on their role. For example:

- **Operations / Product:** Can view and edit merchant configs, publish changes, view logs
- **Production Support:** Can view configs and logs, cannot publish
- **Engineers:** Full access including system-level views and release management
- **Read-only stakeholders:** Dashboard and analytics only

**Business value**

- Allows non-technical teams to use the tool safely — guardrails prevent destructive actions
- Satisfies compliance and audit requirements for who can change production configuration
- Enables the tool to be opened to a wider internal audience without risk

**Engineering requirements**

- Authentication integration with existing SSO provider
- Permission model defined per feature area
- Backend enforcement of permissions (not just UI-level hiding)
- Audit log tied to user identity

---

## 8. Global Stack View (Future Phase)

**What it does**

Today the CRM is scoped to a single stack deployment. In a future phase, it becomes a global control plane — one instance that has visibility and control across all stacks (NA1, NA2, CF, SF, MEG, etc.) from a single interface.

**Business value**

- Eliminates the need to maintain separate deployments per stack
- Gives leadership a true fleet-wide view of the payment application
- Single place to push global changes (e.g. a new feature available to all merchants)

**Engineering requirements**

- Multi-stack API routing and authentication
- Deployment strategy: single global instance vs. federated with global aggregation layer
- Data isolation guarantees per stack (a CF user should not see MEG data)
- This is a significant infrastructure change and is correctly scoped to a later phase

---

## 9. AI Assistance (Future Phase)

**What it does**

AI can be incorporated in several high-value ways:

- **Migration assistant:** Given a new schema version and a merchant's current config, AI suggests the correct migration steps and flags risks
- **Config validation:** AI reviews a proposed config change and flags potential issues before publishing

- **Support assistant:** Production Support describes a client issue in plain language; AI queries logs and config history and surfaces likely causes
- **Documentation generation:** AI generates human-readable summaries of what changed in a migration for client-facing communications

**Business value**

- Multiplies the productivity of a small engineering team
- Makes the tool useful for non-technical users who need guidance, not just data
- Reduces dependency on senior engineer time for routine investigations

**Engineering requirements**

- Integration with Claude API (or similar) for structured reasoning tasks
- Context packaging: the CRM needs to pass relevant merchant config, logs, and schema diff as context
- Rate limiting and cost controls for API usage
- Clear scope boundaries: AI assists, humans approve

---

## Phased Delivery Plan

Given the team size (FE and BE engineers on the payments team) and the goal of a lightweight production release by end of year, the following phasing reflects the real technical dependency chain. Each phase has a defined set of prerequisites that must be resolved before work can begin, and each phase ships independently usable value.

The dependency chain in plain terms:

> *Foundation must ship before anything runs in production → Phase 1 is the first user-facing release → Phase 2 expands access to more teams → Phase 3 activates the release workflow → Phase 4 is scale and intelligence*

Non-code approvals (GitHub API, Coralogix access, SSO wiring) can be pursued in parallel during Foundation work so they do not create sequential delays.

---

### Phase 0 — Foundation

*Internal work. No user-facing features. Two parallel tracks that must both complete before Phase 1 begins.*

Phase 0 has two independent workstreams that can run simultaneously. Neither blocks the other. Both are currently pending approval and are included in the same approval ask as the CRM itself.

---

#### Track A — Backend Services

*Owned by: BE engineer*

- **AccessoPayService aggregation endpoint (read path)** — a new endpoint on AccessoPayService that pulls from the current database tables (ApplicationConfigs, PaymentConfiguration, ApplicationLocale) and returns a single merged payload the CRM can read. AccessoPayService already owns this data; this is a new surface on top of it.
- **Admin API (write path)** — a set of RESTful endpoints that allow the CRM to write admin overlay changes back to the database. These are net-new endpoints designed specifically for this tool, built on AccessoPayService. Covers: update config, publish snapshot, retrieve version history, restore a previous version.

- **Config version snapshot table** — a new database table owned by the payments team that stores a timestamped record of each published config per merchant. No dependency on other teams.
- **SSO wiring and AWS deployment** — the CRM application sits behind the existing VPN and inherits the SSO token authentication already in place. AWS deployment follows the same pipeline used by existing projects. No new infrastructure decisions required.

---

**Track B — Frontend Monorepo Reorganization**

*Owned by: FE engineer*

The accesso-pay-v3 monorepo currently mixes Angular-specific application code with logic that has no reason to be Angular-specific — domain rules, config schemas, API contracts, UI utilities. This track separates them into clearly owned packages with enforced boundaries.

**What gets reorganized:**

- **Shared config schema types** — the TypeScript interfaces that define what a merchant configuration looks like (pages, components, integrations, feature flags, theme). Once extracted as a framework-agnostic package, both accessoPay and the Admin CRM import from the same source. A schema type change is made once and both applications stay in sync automatically.
- **Domain logic** — business rules and validation logic (config merging, feature flag evaluation, integration rules) that currently lives inside the Angular app. Extracting this means the CRM can enforce the same rules without duplicating code.
- **UI component library** — shared visual components (buttons, form inputs, cards, badges) available to any application in the monorepo. The CRM's interface can match the payment application's look and feel without building from scratch.
- **UI utilities** — formatters, helpers, and common functions reusable across applications.

**Why this matters for the CRM specifically:**

Without this reorganization, the CRM would need to maintain its own copy of config type definitions, validation rules, and UI components — creating two sources of truth that drift apart over time. Any schema change would require updates in two places. The reorganization eliminates that class of problem entirely by making the CRM a first-class consumer of the same shared packages the payment application uses.

**Why this matters beyond the CRM:**

The reorganization benefits every current and future frontend application in the payments domain. It is not CRM-specific overhead — it is foundational work that pays dividends across the entire frontend portfolio. This makes the investment easier to justify: it serves the existing application, the CRM, and any future tooling built in the same monorepo.

---

**Track C — accessoPay Initialization Refactor**

*Owned by: FE engineer · Can run in parallel with Tracks A and B*

accessoPay currently builds its pages based on hardcoded merchant app configs and feature flags. There is no schema that drives layout rendering — which page appears in which order, which components are shown on each page, and which third-party services are initialized are all determined by logic baked into the application itself.

For the CRM's layout and component controls to have real effect in the live checkout experience, accessoPay needs to be refactored to:

- **Adopt the new layout schema** — read pages, components, visibility, and ordering from the schema the CRM produces, rather than from hardcoded application logic
- **Dynamic initialization of third-party services** — load and initialize payment providers, fraud detection tools, insurance services, and other integrations based on the schema's integration config and their defined initialization strategy (critical, prefetch, lazy), rather than hardcoded startup logic

**What Phase 1 can deliver without this refactor:**

Not all CRM controls depend on the accessoPay refactor. The following work immediately through AccessoPayService and existing configuration mechanisms:

- Theme and branding changes (primary color, fonts, border radius)
- Integration toggles (enable/disable a payment provider)
- Feature flags

The following controls are fully built and visible in the CRM in Phase 1 but do not take effect in the live accessoPay application until Track C is complete:

- Page visibility and ordering
- Component visibility and ordering within a page
- Component-level feature configuration

This means Phase 1 ships with real production value. Track C completing in a subsequent iteration unlocks the full effect of the layout controls already built.

**Why this is the correct sequencing:**

Attempting to do the accessoPay refactor before the CRM exists would mean refactoring toward a schema with no tooling to manage it. The CRM and the schema are built together first, then accessoPay is refactored to consume them. The preview mockup in Phase 1 shows exactly what the experience will look like once the refactor is in place — giving the team and stakeholders a concrete reference point.

---

**What Phase 0 unlocks:** Phase 1 can begin as soon as Tracks A and B are complete. Track C can be executed as a Phase 1 parallel workstream or as an immediate follow-on iteration. Coralogix API access, GitHub API approval, and parent app version API outreach should all be initiated during Phase 0 so they are ready when needed.

---

## Phase 1 — Config Management & Preview

*First production release. Target: end of year.*

**What gets built:**

- **Merchant configuration editor** — visual controls for theme and branding, page visibility, component toggles, integration settings, and feature flags. Reads from the Phase 0 aggregation endpoint; writes through the Phase 0 admin API. Theme, integration, and feature flag controls take effect immediately. Page and component layout controls take full effect once the accessoPay initialization refactor (Track C) is complete.
- **Publish and version history** — every saved change creates a snapshot. Users can browse the full history for any merchant and restore any previous version. Stored in the Phase 0 snapshot table.
- **Preview mockup** — a visual representation of how the checkout experience will look with the current configuration, generated by the CRM in real time. Does not require the accessoPay refactor. Serves as the reference view for what Track C will produce in the live application.
- **Basic role-based access** — read-only vs. edit permissions, inherited from the existing internal user role service via SSO. Backend enforced on the admin API.

- **Per-stack deployment** — deployed once per stack on existing AWS infrastructure.

**Technical dependencies:** Phase 0 Tracks A and B complete

**Who does the work:** FE engineer (config editor UI, preview, version history views) + BE engineer (API integration, RBAC backend enforcement)

**What this delivers for the business:** Operations and Product can enable and adjust merchant configurations without SQL deployment tasks or engineering involvement. Every change is tracked, reversible, and safe to execute without a skilled engineer present. This is the core ROI of the entire project.

---

### Phase 2 — Observability & Access Control

*Expands the tool to Production Support and a wider Ops audience.*

**What gets built:**

- **Logs viewer** — searchable, filterable log view connected to the existing Coralogix instance via its API. Scoped to payment-domain events: config publish events, integration activity, system errors. No new logging infrastructure needed.
- **Analytics dashboard** — high-level view of configuration state across merchants and stacks, derived from config data already available after Phase 1. No additional data sources required at this stage.
- **Full role enforcement** — expands the Phase 1 basic access model to the full role set: Product/Operations (edit + publish), Production Support (view-only), Engineers (full access). Role definitions inherited from the existing user role service.
- **Publish audit trail** — every publish event is tied to the authenticated user's identity, creating a complete record of who changed what and when.

**Technical dependencies:** Phase 1 stable in production; Coralogix API access approved (request during Phase 0)

**Who does the work:** FE engineer (logs UI, analytics views) + BE engineer (Coralogix API integration, role enforcement on existing endpoints)

**What this delivers for the business:** Production Support can triage client issues without pulling in an engineer. Operations has a complete audit trail. The tool is now useful to non-engineers day-to-day, not just when making a config change.

---

### Phase 3 — Release & Migration Center

*Eliminates manual investigation work every time the payment application ships a new version.*

**What gets built:**

- **GitHub release sync** — the CRM fetches releases from the accessoPay GitHub repository via the GitHub API. Engineers use a structured release template (already built and documented) when cutting each version. The CRM parses this into a human-readable changelog automatically.
- **Per-merchant version tracking** — the CRM displays which version of accessoPay each merchant is currently running. This data is sourced from parent application APIs. Each parent app that embeds accessoPay exposes an endpoint that provides the active version for a given merchant. Outreach to parent app teams to establish these API contracts must begin during Phase 0 — this is an external dependency with lead time that cannot be deferred.
- **Per-merchant migration status** — using the version data above, the CRM shows which merchants are on which version and what changes are required to reach the latest. Stored in a new payments-

team-owned database table.
- **Guided migration steps** — for each merchant that needs migration, the CRM presents a checklist of the specific changes required based on the diff between their current version and the target version.

**Immediate need note:** Knowing which version of accessoPay each merchant is running is already an operational need independent of the full Release Center. The parent app API outreach should be treated as a parallel discovery item starting now, not gated on Phase 3 beginning.

**Technical dependencies:** Phase 2 complete; GitHub API access approved (request during Phase 0); parent app version API contracts established (outreach during Phase 0)

**Who does the work:** FE engineer (release UI, version tracking views, migration status) + BE engineer (GitHub API integration, parent app API integration, migration tracking table, version diff logic)

**What this delivers for the business:** No more manually reaching out to find out which merchants are on which version. No more manual investigation when accessoPay ships a new version. The team knows immediately which merchants need migration, what exactly needs to change, and can work through it systematically rather than from memory.

---

### Phase 4 — Scale & Intelligence

*Highest complexity. Planned based on what Phase 1–3 teaches us in production.*

**What gets built:**

- **Global stack view** — a single CRM instance with visibility and control across all stacks (NA1, NA2, CF, SF, MEG, etc.). This is an infrastructure change: multi-stack API routing, data isolation guarantees, and a deployment strategy decision (single global instance vs. federated). This is the correct scope for a later phase because it requires operational stability first.
- **Live preview** — opens a real instance of accessoPay loaded with the draft config, scoped to the specific tenant and merchant. Requires a mechanism to be added to the accessoPay for accepting CRM-controlled config. Designed in coordination with the accessoPay team.
- **AI assistance** — integrated AI for migration guidance, config validation before publish, and support query resolution. Uses existing AI API infrastructure. Specific capabilities are scoped based on what the team has learned from real usage in Phases 1–3.
- **Advanced analytics** — transaction and conversion data connected to config state. Enables comparison of business outcomes across different configuration choices.

**Technical dependencies:** Phase 3 complete; multi-stack infrastructure planning; accessoPay coordination for live preview; AI API access

**Who does the work:** Full payments team + infrastructure planning + accessoPay team coordination

**What this delivers for the business:** One control panel for the entire fleet. AI removes the remaining manual work. Leadership gets a true portfolio view of payment performance across all clients and configurations.

---

## Addressing Expected Concerns

### "This is too costly."

The cost of *not* building it is already being paid — in engineer hours spent on configuration tickets, sprint cycles consumed by routine client requests, and incident time spent manually correlating data across

systems. This tool converts that ongoing operational cost into a one-time build investment. With AI-assisted development, the build cost is lower than it would have been two years ago.

### "Our team is only 2 engineers — we don't have the capacity."

This is precisely the argument *for* the tool. A 2-person team cannot scale manually to the demands of a growing merchant fleet. The CRM creates leverage: it takes routine work off engineers entirely and hands it to the people who own those relationships. Phase 1 alone removes a significant class of tickets from the engineering backlog.

### "We have other tools that do similar things."

Existing tools are not built around the payment application's schema, stacks, or integration partners. Using a generic tool requires engineers to translate between the tool's model and the application's model — which is its own source of error and overhead. A dedicated tool means zero translation cost and can enforce application-specific validation rules that a generic tool cannot.

### "What if it takes longer than expected?"

Each phase is independently shippable. If Phase 1 takes longer, the team ships Phase 1 and the business gets value immediately. Nothing in the phased plan requires all phases to succeed for Phase 1 to deliver ROI.

---

## What We're Asking For

There are three items currently pending approval. They are related and benefit from being approved together, but each has standalone value if approved individually.

1. **Monorepo reorganization** — separate Angular-specific code from framework-agnostic packages (SDK, domain logic, config schemas, UI component library). This is Phase 0 Track B. It benefits the existing payment application immediately and is a prerequisite for the CRM to be built cleanly. Research and task items are already defined.
2. **Payments Admin CRM — Phase 0 & Phase 1** — approve Foundation work (Backend API design + CRM project scaffolding) and the first production release (config management, preview, version history). Target: lightweight production release by end of year.
3. **Recognition that this is a roadmap item** — not a side project. The payments team is a 2-person FE + BE team. For these projects to ship this year, they need to be on the sprint roadmap with protected capacity, not worked around other priorities.

**No additional headcount required.** The existing payments team executes all of this with AI-assisted development. The ask is time and approval, not resources.

---

## Summary

| Area | Today | With Admin CRM |
|---|---|---|
| Client config change | Weeks (full SDLC) | Minutes (self-service) |
| Incident investigation | Manual, multi-system | Single pane, searchable |
| Migration tracking | Manual, undocumented | Automated, per-merchant status |

| | | |
|---|---|---|
| Configuration confidence | Low — no history | High — version history + rollback |
| Audience | Engineers only | Product, Ops, Support |
| Fleet visibility | None | Dashboard across all merchants |
| Engineering ticket load | High | Significantly reduced |

The POC demonstrates that the architecture works and the experience is achievable. The ask is to make it real.